

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
- (b) Design a responsive navigation bar using appropriate CSS techniques.
- (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).

(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
 - (b) Suggest appropriate **CSS animation or transition** for product interaction.
 - (c) Design a hover effect for a product card.
 - (d) Evaluate performance and user experience considerations.
-

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).
 - (b) Compare **Flexbox and Grid** based on layout needs.
 - (c) Choose the most suitable layout technique and justify your choice.
 - (d) Evaluate the scalability and responsiveness of the chosen approach.
-

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
 - (b) Design a mobile-first layout using CSS.
 - (c) Demonstrate how the layout scales for larger screens using media queries.
 - (d) Evaluate accessibility and readability of the design.
-

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

I K PHANI SRIDHAR YOGI 192311091 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

A handwritten signature in blue ink, reading "K-P-Sridhar" with a horizontal line underneath.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis

Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation
--------------------------------	----	---	--------------------------------	--------------------------------	-------------------------------

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.**
- (b) Design a responsive navigation bar using appropriate CSS techniques.**
- (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).**
- (d) Evaluate the usability and suggest improvements for better user experience.**

(a) Identify key components required in a navigation bar for an e-commerce website.

An e-commerce navigation bar should provide easy access to the main features of the website. The important components include:

- **Logo/Brand Name:** Identifies the e-commerce website and links to the home page.
- **Home:** Allows users to quickly return to the main page.
- **Products/Categories:** Helps users browse products based on different categories.
- **Search Bar:** Enables users to quickly find specific products.
- **Shopping Cart:** Displays selected products and the number of items added.
- **Login/Register:** Allows customers to create accounts and access their profiles.
- **Wishlist:** Enables users to save products for future purchases.
- **User Account:** Provides access to orders, profile details, and account settings.

A well-designed navigation bar should be simple, consistent, visually clear, and easy to access on all screen sizes.

(b) Design a responsive navigation bar using appropriate CSS techniques.

A responsive navigation bar can be designed using HTML, CSS Flexbox, and CSS Media Queries. Flexbox is useful for arranging the logo, navigation links, search bar, and user actions in a single row. The justify-content and align-items properties can be used to maintain proper alignment and spacing.

CSS media queries can be used to change the layout according to the device width. On desktop screens, navigation items can be displayed horizontally, while on tablets the spacing and font sizes can be reduced. The design should use relative units such as %, rem, and vw instead of fixed dimensions wherever possible.

For example, the basic structure can be:

```
<nav class="navbar">

  <div class="logo">ShopEasy</div>


  <div class="search">

    <input type="text" placeholder="Search products">

  </div>


  <ul class="nav-links">

    <li><a href="#">Home</a></li>

    <li><a href="#">Products</a></li>

    <li><a href="#">Categories</a></li>

    <li><a href="#">Cart</a></li>

    <li><a href="#">Login</a></li>

  </ul>


  <button class="menu-btn">☰</button>

</nav>
```

The CSS can use Flexbox to create a flexible layout:

```
.navbar {
```

```
display: flex;
align-items: center;
justify-content: space-between;
padding: 15px 5%;
}
```

```
.nav-links {
display: flex;
gap: 20px;
list-style: none;
}
```

```
.menu-btn {
display: none;
}
```

This approach provides a well-structured and flexible navigation layout.

(c) Demonstrate how the navigation adapts to smaller screens.

For mobile devices, the horizontal navigation links may occupy too much space. Therefore, a hamburger menu ($\equiv$) can be introduced. Using a CSS media query, the normal navigation links are hidden and the hamburger button is displayed.

```
@media (max-width: 768px) {
.nav-links {
display: none;
flex-direction: column;
width: 100%;
}
```

```
}

.menu-btn {
    display: block;
}

.navbar {
    flex-wrap: wrap;
}
}
```

JavaScript can be used to open and close the mobile menu when the hamburger icon is clicked:

```
document.querySelector(".menu-btn").onclick = function() {
    document.querySelector(".nav-links").classList.toggle("active");
};
```

Thus, on a desktop, users can see all navigation links in a horizontal layout. On a tablet, the layout adjusts its spacing and size. On a mobile device, the links are replaced by a hamburger menu, reducing screen clutter and making navigation easier.

(d) Evaluate the usability and suggest improvements for better user experience.

The responsive navigation bar improves usability by providing a consistent navigation experience across desktop, tablet, and mobile devices. The search bar allows customers to find products quickly, while the cart and account options provide easy access to important shopping functions. The hamburger menu prevents the mobile screen from becoming overcrowded.

The following improvements can provide a better user experience:

1. Use a sticky navigation bar so that users can access navigation while scrolling.
2. Use clear icons for search, cart, wishlist, and user accounts.
3. Ensure all buttons and menu items are large enough for touch interaction on mobile devices.

4. Maintain good color contrast between text and background for readability.
5. Provide hover and active effects to indicate the selected navigation item.
6. Make the hamburger menu easy to open and close.
7. Optimize the navigation for fast loading and accessibility.
8. Ensure the layout does not overlap or break when the browser window is resized.

Overall, a responsive navigation bar designed using Flexbox, CSS media queries, and JavaScript provides a flexible and user-friendly interface. Proper testing on different screen sizes is necessary to ensure that the navigation remains aligned, accessible, responsive, and easy to use.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.**
- (b) Suggest appropriate CSS animation or transition for product interaction.**
- (c) Design a hover effect for a product card.**
- (d) Evaluate performance and user experience considerations**

Question 2: Product Hover Effect – 4 Marks

- (a) Identify suitable CSS properties for hover effects.**

CSS hover effects are used to provide visual feedback when a user moves the mouse pointer over a product. Suitable CSS properties include:

- transform – Used to scale, rotate, or move the product card.
- box-shadow – Adds depth and makes the product card visually stand out.
- background-color – Changes the background color when hovering.
- opacity – Controls the transparency of elements.
- border – Changes or highlights the border of the product card.
- filter – Can be used to adjust brightness or other visual effects.
- transition – Makes the change between normal and hover states smooth.

These properties can be combined to create an attractive and interactive product card.

- (b) Suggest appropriate CSS animation or transition for product interaction.**

A CSS transition is suitable for simple product hover interactions because it provides smooth visual changes without requiring complex animation. For example, when the user hovers over a product card, the card can slightly increase in size and display a shadow.

The following transition can be used:

```
.product-card {  
    transition: transform 0.3s ease,  
                box-shadow 0.3s ease;  
}
```

The transform property can be used with scale() to slightly enlarge the card:

```
.product-card:hover {  
    transform: scale(1.05);  
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);  
}
```

A duration of approximately 0.2 to 0.3 seconds provides a smooth response without making the interface feel slow. For more complex effects, CSS @keyframes animations can be used, but simple transitions are generally more appropriate for product cards.

(c) Design a hover effect for a product card.

A product card can contain the product image, product name, price, description, and Add to Cart button. When the user moves the cursor over the card, it can slightly rise, enlarge, and display a shadow. The product image can also become slightly brighter.

Example:

```
<div class="product-card">  
      
    <h3>Smart Watch</h3>  
    <p>Latest Smart Watch</p>  
    <p class="price">$49.99</p>  
    <button>Add to Cart</button>  
</div>  
  
.product-card {
```

```
width: 250px;
padding: 20px;
text-align: center;
border: 1px solid #ddd;
border-radius: 10px;
background: white;
transition: transform 0.3s ease,
            box-shadow 0.3s ease;
}
```

```
.product-card img {
width: 100%;
transition: transform 0.3s ease,
            filter 0.3s ease;
}
```

```
.product-card:hover {
transform: translateY(-8px);
box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
}
```

```
.product-card:hover img {
transform: scale(1.05);
filter: brightness(1.1);
}
```

```
.product-card button {  
    transition: background-color 0.3s ease;  
}
```

```
.product-card button:hover {  
    background-color: #333;  
    color: white;  
}
```

This design creates a smooth interaction where the product card moves upward, the image slightly zooms, and a shadow appears when the user hovers over the product. This helps the selected product stand out from other products.

(d) Evaluate performance and user experience considerations.

Hover effects can improve the user experience by providing immediate visual feedback and helping users identify which product they are interacting with. A subtle scale or shadow effect can make the website feel more interactive and attractive.

However, excessive animations should be avoided because they can distract users and negatively affect performance. The following considerations should be followed:

1. Use short transitions of around 0.2–0.3 seconds for quick responses.
2. Avoid excessive animations that may distract users from product information.
3. Prefer CSS transitions and transforms because they generally provide smooth performance.
4. Avoid unnecessarily large images and optimize product images to improve page loading speed.
5. Ensure hover effects do not cause layout shifting or overlapping.
6. Maintain sufficient color contrast for readability.
7. Provide alternative visual feedback for touchscreen and keyboard users, since hover is not available in the same way on mobile devices.

8. Respect users who prefer reduced motion by using the prefers-reduced-motion media query when appropriate.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).**
- (b) Compare Flexbox and Grid based on layout needs.**
- (c) Choose the most suitable layout technique and justify your choice.**
- (d) Evaluate the scalability and responsiveness of the chosen approach.**

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

A dashboard usually contains multiple sections such as a header, sidebar, navigation menu, main content area, statistics cards, charts, tables, and footer. Therefore, the layout should support both rows and columns.

The main layout requirements are:

- Header: A horizontal row containing the application logo, title, notifications, and user profile.
- Sidebar: A vertical column containing navigation links and menu options.
- Main Content: Displays dashboard information using multiple rows and columns.
- Cards: Statistics such as total users, sales, orders, and revenue can be arranged in a row or grid.
- Charts and Tables: Require flexible columns with proper alignment.
- Alignment: Elements should be aligned consistently using properties such as align-items, justify-content, and gap.
- Responsiveness: The layout should automatically adjust when viewed on desktop, tablet, and mobile screens.

Thus, the dashboard requires a two-dimensional layout system that can manage both rows and columns effectively.

(b) Compare Flexbox and Grid based on layout needs.

Flexbox and CSS Grid are both powerful CSS layout techniques, but they are suitable for different requirements.

Feature	Flexbox	CSS Grid
Layout type	One-dimensional	Two-dimensional
Main direction	Row or column	Rows and columns simultaneously
Best suited for	Navigation bars, menus, and small components	Complete dashboard layouts
Alignment	Excellent for aligning items in a row or column	Excellent for row and column alignment
Responsive design	Flexible and easy to implement	Supports responsive layouts using grid techniques
Complexity	Simple for linear layouts	Better for complex layouts
Dashboard suitability	Useful for individual components	Highly suitable for overall dashboard structure

Flexbox is ideal when elements need to be arranged in a single direction, such as a navigation bar or a row of buttons. CSS Grid is more suitable when the dashboard requires control over both rows and columns.

(c) Choose the most suitable layout technique and justify your choice.

For a complete dashboard layout, CSS Grid is the most suitable technique because a dashboard generally has a two-dimensional structure consisting of multiple rows and columns.

For example, the dashboard can be structured as:

```
.dashboard {  
  display: grid;  
  grid-template-columns: 220px 1fr;  
  grid-template-rows: 70px 1fr;  
  min-height: 100vh;  
  gap: 10px;  
}
```

```
.header {  
  grid-column: 1 / 3;  
}
```

```
.sidebar {  
  grid-column: 1;  
}
```

```
.main-content {  
  grid-column: 2;  
}
```

In this design, CSS Grid creates a fixed sidebar and a flexible main content area. The header can span across the required columns, while the main content can contain additional grid layouts for cards, charts, and tables.

For example:

```
.dashboard-cards {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 20px;
```

```
}
```

Therefore, CSS Grid is selected as the primary layout technique because it provides better control over the overall dashboard structure. Flexbox can still be used inside individual components, such as arranging items within a navigation bar, card, or header.

(d) Evaluate the scalability and responsiveness of the chosen approach.

CSS Grid provides good scalability and responsiveness because the number and size of columns can be adjusted according to the available screen width. The `fr` unit, `minmax()`, `auto-fit`, and `auto-fill` properties allow dashboard components to resize automatically.

For example:

```
.dashboard-cards {  
    display: grid  
  
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  
    gap: 20px;  
}
```

This allows the cards to automatically fit the available screen space. On a large desktop screen, several cards can appear in one row. On tablets, fewer cards may appear per row, while on mobile devices, the cards can automatically move into a single-column layout.

Media queries can also be used:

```
@media (max-width: 768px) {  
    .dashboard {  
        grid-template-columns: 1fr;  
    }  
  
    .sidebar {  
        display: none;  
    }  
  
    .main-content {
```

```
    grid-column: 1;
}
}
```

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.**
- (b) Design a mobile-first layout using CSS.**
- (c) Demonstrate how the layout scales for larger screens using media queries.**
- (d) Evaluate accessibility and readability of the design.**

(a) Identify key sections of a blog layout.

A blog website generally consists of several important sections that help users read and navigate content easily. The key sections include:

- **Header:** Contains the blog logo, website title, and navigation menu.
- **Navigation Bar:** Provides links to Home, Categories, About, and Contact pages.
- **Main Content:** Displays the blog title, author information, publication date, featured image, and article content.
- **Sidebar:** Contains recent posts, categories, search, tags, or advertisements on larger screens.
- **Footer:** Contains copyright information, social media links, and additional navigation.
- **Comments Section:** Allows readers to interact with the author and other users.

The layout should be organized clearly so that the main article remains the primary focus, especially on mobile devices.

(b) Design a mobile-first layout using CSS.

A mobile-first approach means designing the website for smaller screens first and then adding styles for larger screens. The basic mobile layout should use a single-column structure because mobile screens have limited width.

Example HTML:

```
<div class="blog-container">
```

```
  <header>
```

```
    <h1>My Tech Blog</h1>
```

```
    <nav>
```

```
      <a href="#">Home</a>
```

```
      <a href="#">Articles</a>
```

```
      <a href="#">About</a>
```

```
      <a href="#">Contact</a>
```

```
    </nav>
```

```
  </header>
```

```
  <main>
```

```
    <article>
```

```
      <h2>Introduction to Web Design</h2>
```

```
      <p class="meta">By Sai | July 29, 2026</p>
```

```
      
```

```
      <p>
```

```
        Web design focuses on creating attractive,  
        accessible, and responsive websites.
```

```
      </p>
```

```
    </article>
```

```
  <aside>
```

```
<h3>Recent Posts</h3>
```

```
<p>Responsive Web Design</p>
```

```
<p>Introduction to CSS</p>
```

```
</aside>
```

```
</main>
```

```
<footer>
```

```
<p>© 2026 My Tech Blog</p>
```

```
</footer>
```

```
</div>
```

Mobile-first CSS:

```
* {  
  box-sizing: border-box;  
}
```

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
}
```

```
.blog-container {  
  width: 100%;  
  padding: 15px;  
}
```

```
header {  
    text-align: center;  
}
```

```
nav {  
    display: flex;  
    flex-direction: column;  
    gap: 10px;  
}
```

```
nav a {  
    text-decoration: none;  
    padding: 8px;  
}
```

```
main {  
    display: flex;  
    flex-direction: column;  
    gap: 20px;  
}
```

```
article img {  
    width: 100%;  
    height: auto;  
}
```

```
aside {  
  padding: 15px;  
}
```

This design starts with a single-column layout, making it easy to read and navigate on mobile devices.

(c) Demonstrate how the layout scales for larger screens using media queries.

Media queries can be used to modify the layout when the screen becomes wider. On tablets and desktops, the article and sidebar can be displayed side-by-side.

```
@media (min-width: 768px) {
```

```
  .blog-container {  
    max-width: 1100px;  
    margin: auto;  
  }
```

```
  nav {  
    flex-direction: row;  
    justify-content: center;  
    gap: 25px;  
  }
```

```
  main {  
    display: grid;  
    grid-template-columns: 2fr 1fr;  
    gap: 30px;  
  }
```

```
article {  
    padding: 20px;  
}
```

```
aside {  
    padding: 20px;  
}  
}
```

On mobile devices, the content appears in a single column. On tablet and desktop screens, CSS Grid creates two columns, with the blog article receiving more space than the sidebar. This approach allows the layout to scale smoothly across different screen sizes.

(d) Evaluate accessibility and readability of the design.

Accessibility and readability are important for ensuring that all users can comfortably access blog content. The following practices can improve the design:

1. Use semantic HTML elements such as <header>, <nav>, <main>, <article>, <aside>, and <footer>.
2. Provide meaningful alt text for images so screen readers can describe them.
3. Use sufficient color contrast between text and background.
4. Select a readable font and maintain an appropriate font size and line height.
5. Use proper heading levels such as <h1>, <h2>, and <h3> to organize content.
6. Ensure navigation links are accessible using keyboard controls.
7. Make buttons and links large enough for easy touch interaction.
8. Avoid excessive text width on desktop screens to improve reading comfort.
9. Use responsive images that do not overflow the screen.
10. Test the page using different devices and screen sizes.

Overall, the mobile-first approach ensures that the blog is fast, readable, accessible, and user-friendly on small screens while media queries allow it to expand effectively for tablets

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

(b) Design event handling logic for menu toggle functionality.

(c) Implement interaction for user actions (click/hover).

(d) Evaluate performance and suggest improvements.

(a) Identify appropriate JavaScript events for menu interaction.

JavaScript events allow a dynamic menu to respond to user actions. The appropriate events include:

- click – Used to open or close a menu when the user clicks a hamburger button.
- mouseover – Detects when the mouse pointer moves over a menu item.
- mouseout – Detects when the pointer leaves a menu item.
- mouseenter and mouseleave – Useful for controlling dropdown menus without triggering events on child elements.
- keydown – Allows keyboard users to interact with the menu.
- focus and blur – Useful for keyboard accessibility and focus management.

For a mobile menu, the click event is generally the most appropriate because mobile devices do not support hover interactions in the same way as desktop devices.

(b) Design event handling logic for menu toggle functionality.

The menu toggle functionality can be implemented using a hamburger button. When the user clicks the button, JavaScript adds or removes an active class from the navigation menu.

HTML:

```
<button id="menuBtn" aria-label="Open navigation menu">
```

```
  ≡
```

```
</button>
```

```
<nav id="menu">
```

```
  <a href="#">Home</a>
```

```
  <a href="#">Products</a>
```

```
  <a href="#">Services</a>
```

```
  <a href="#">Contact</a>
```

```
</nav>
```

CSS:

```
#menu {
```

```
  display: none;
```

```
  flex-direction: column;
```

```
}
```

```
#menu.active {
```

```
  display: flex;
```

```
}
```

JavaScript:

```
const menuBtn = document.getElementById("menuBtn");
```

```
const menu = document.getElementById("menu");
```

```
menuBtn.addEventListener("click", function() {
```

```
  menu.classList.toggle("active");
```

```
});
```

The `classList.toggle()` method automatically adds the active class when the menu is closed and removes it when the menu is open. This provides simple and efficient menu control.

(c) Implement interaction for user actions (click/hover).

A dynamic menu can support both click and hover interactions. The click event is mainly useful for mobile devices, while hover effects can improve desktop navigation.

Example:

```
const menuItems = document.querySelectorAll(".menu-item");
```

```
menuItems.forEach(function(item) {  
  
    item.addEventListener("mouseenter", function() {  
        item.classList.add("hover");  
    });  
  
    item.addEventListener("mouseleave", function() {  
        item.classList.remove("hover");  
    });  
});
```

CSS:

```
.menu-item.hover {  
    background-color: #ddd;  
    transform: translateY(-2px);  
}
```

For a dropdown menu, JavaScript can also toggle a submenu:

```
const dropdownBtn = document.querySelector(".dropdown-btn");  
const submenu = document.querySelector(".submenu");
```

```
dropdownBtn.addEventListener("click", function() {  
    submenu.classList.toggle("show");  
});
```

This provides interactive behavior when users click the menu. On desktop devices, hover can provide quick visual feedback, while on mobile devices, clicking is more reliable.

(d) Evaluate performance and suggest improvements.

JavaScript event handling should be implemented efficiently to provide a smooth user experience. The following improvements can be considered:

1. Use `addEventListener()` instead of inline JavaScript event handlers for better code organization.
2. Use CSS classes to control visual changes rather than directly changing many CSS properties through JavaScript.
3. Avoid attaching unnecessary event listeners to a large number of elements.
4. Use event delegation when multiple menu items have similar interactions.
5. Keep event handlers simple to reduce processing time.
6. Use click events for mobile navigation instead of relying only on hover.
7. Support keyboard navigation using `keydown`, `focus`, and `blur` events.
8. Use ARIA attributes such as `aria-expanded` to communicate menu state to assistive technologies.
9. Ensure the menu does not cause unnecessary layout shifts.
10. Test the menu on desktop, tablet, and mobile devices.

For example, the accessibility state can be updated as follows:

```
menuBtn.addEventListener("click", function() {  
    const isOpen = menu.classList.toggle("active");
```

```
menuBtn.setAttribute(  
    "aria-expanded",  
    isOpen  
);  
});
```

Overall, using JavaScript event listeners with CSS classes provides an efficient and maintainable solution for a dynamic menu. Combining click-based interaction for mobile devices, hover effects for desktop users, and keyboard accessibility creates a responsive and user-friendly navigation system.